Requirements Analysis Document

Document Written By: Tristan Austin Niko Hoogeveen Yexin Jin Camila Izquierdo Ben Graham Ian Keefe Nicholas Eto

Background and History

C++ is a general-purpose programming language, invented by Bjarne Stroustrup. It is an extension of the C programming language, sometimes referred to as "C with classes". The language will be used to implement a sudoku solving application. The name Sudoku comes from Japan, with characters "su" (meaning number) and "doku" (meaning single), but the game itself was invented in Switzerland by the great mathematician Leonard Euler.

Sudoku is a simple game with a couple rules:

- Every square has to contain a single number
- Only the numbers 1 through 9 can be used
- Each 3x3 box can only contain each number from 1 to 9 once
- Each vertical column can only contain each number from 1 to 9 once
- Each horizontal row can only contain each number from 1 to 9 once

The sudoku application will include a game mode, where the user can play sudoku games of their desired difficulty, and a solving mode where the user can input their sudoku game and compute the solved board.

Purpose and Scope

The general purpose of the sudoku solver program is to create a streamlined, accessible, and easy-to-use application to solve or assist the user to solve a sudoku puzzle using C++; a programming language.

In order to make the application as user-friendly as possible, the game will have a feature to scan a puzzle through a smartphone. This will be a lot easier and more effective compared to input the numbers manually into the application. Manually entering the puzzle will also be an option, should the user desire. After the puzzle is loaded into the program, the user can choose if they want hints on how to solve it (i.e. the computer will tell them one additional correctly placed number at a time) or the completed solution. The computer will prompt the user if the puzzle loaded is un-solvable.

In addition to the s*olve* mode, there is also the *play* mode, where the user can also use the application to play sudoku. This option will generate a random, solvable, sudoku board for the user of specified difficulty (easy, medium, hard). For example, an easy board will give the user more numbers on the sudoku board to make it easier for the user to find the remaining numbers. It will also allow the user to undo a move or erase a move if needed. The user can request hints while solving the puzzle, similar to if they had loaded an external puzzle as described above. The *play* mode will include some additional features including a timed mode, where the user is given a time limit to complete a problem. The user can also unlock new themes and points by completing more sudokus without using hints from the computer.

Objectives and Success Criteria

The objective of this application is to generate a puzzle that needs players to fill in numbers or letters (numbers from one to nine for 9x9 boards and additional letters from A to G for 16x16 boards) into a grid to make every number appears once in each column, row and square. The minimum viable product is to provide players with 9 by 9 boards with different difficulty levels (easy, medium and hard) that varies from the amount and arrangements of the numbers generated. A login page for users to visit, register or log in will be available. The timer starts as the users start the game. Notes can be taken and erased to help users keep better track of their thoughts. Every time the users enter a number, the value will be saved and check whether it is the right answer to that grid. Players could pause the game as they wish with all the values saved. Undo actions can be achieved if the users have entered a wrong number. Once the mistake limit has been reached or the time has run out, the game is over. Hints are available if the users get stuck. In the settings, users can learn how to play Sudoku if they are not familiar with this game. Users are allowed to customize different functions such as regular fonts and themes, mistake limit, time limit sound effects and highlight for identical numbers throughout the grids that help users solve the puzzle faster.
Once the minimum viable product is accomplished, additional functionality could be taken into account accordingly. For instance, a different mode that users scan by phones or manually enter their Sudoku board. The board entered will be checked first to ensure it is solvable. Moreover, the users are possible to unlock new themes or medals by breaking their own or global records or completing more sudokus without using hints.

Definitions and Acronyms

This section will better define some of the language used throughout this report.

Note: Users can note possible solutions to one specific gird or the numbers lacking for each row, column or square.

Highlight areas: Users can set whether to highlight the square, column and row for a selected cell.

Highlight identical numbers: As users select one specific number, the same numbers will be highlighted throughout the grid.

Hide used number: Players can choose to hide numbers that have been placed into the grid correctly.

Mistake limit: Users can customize the mistake limit for one game or simply choose to not set a mistake limit.

Auto-Check for mistake: Mark numbers that do not match the solution to the board as users enter.

Auto-Remove notes: Remove a number from all notes when the number has been entered correctly into its row, column and square.

General Description of The System

There are three main components of the system; GUI, Sudoku Class, and Database.

Upon starting the application the user will be displayed a login screen GUI and prompted to login/sign up. The data base will need to be accessed to carry this out (add new user, check if user's credentials are valid).

Once the user completes login/sign up, a main menu GUI will be displayed with a couple different buttons; New Game, Profile, Solver, Settings, Quit Game.

New Game - user will be able to choose what difficulty of game they want to play. A new game will be generated and the user will have the ability to input numbers in the empty squares. A hint function will be available, and it will provide the user with the next best number to input.

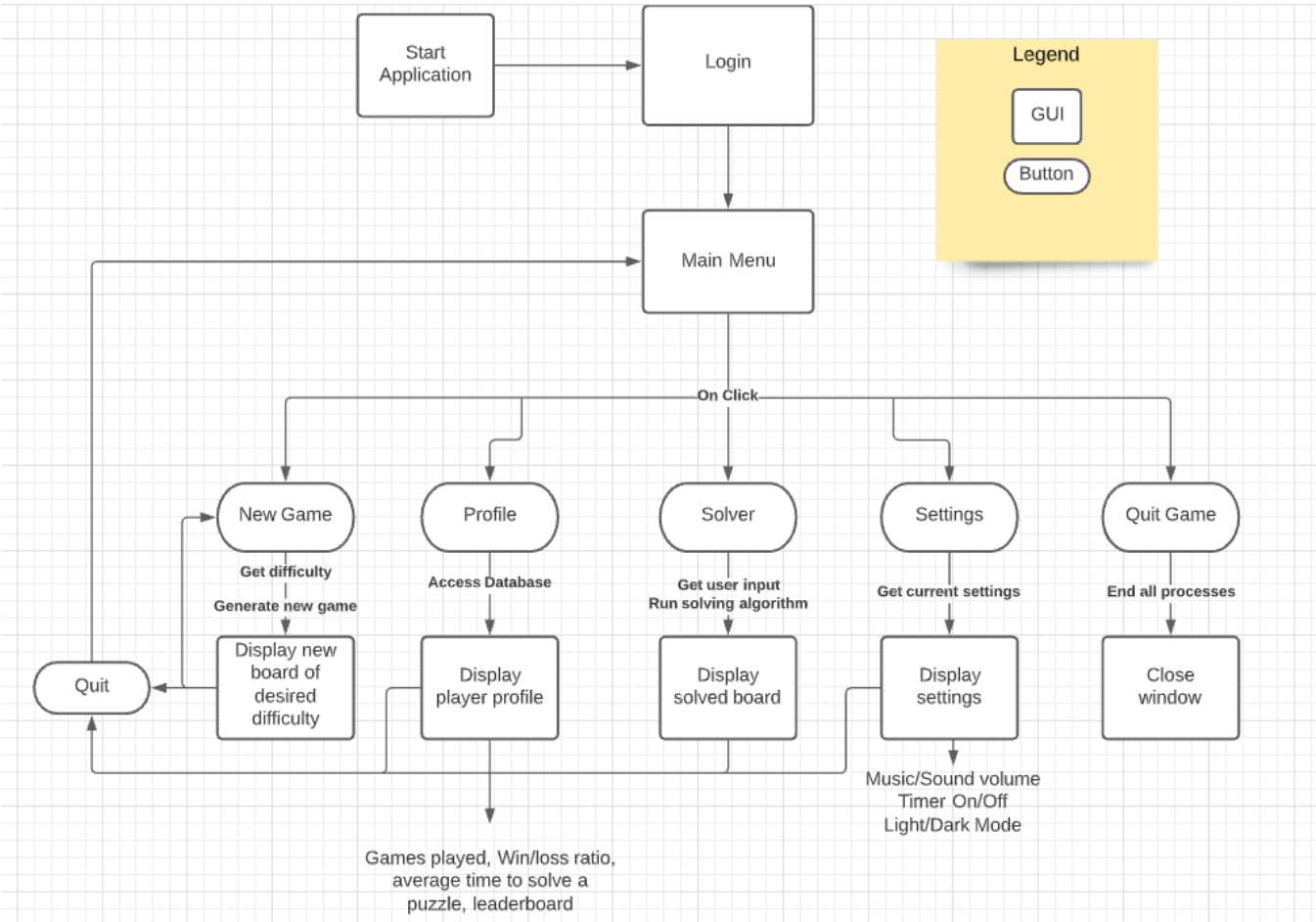
Profile - a GUI will be displayed with statistics of the users account. They will be able to see information such as average time taken to finish a game, game completion rate, highest score of a game (score is calculated depending on difficulty and time taken to finish a game), and the leaderboard of other users high scores.

Solver - the user will be prompted to input the board they want to solve. They will have the choice to input a picture of a board or manually type in the numbers. Once the board has been inputted, the solving algorithm will be run and the solved board will be displayed (if a solution is possible).

Settings - a GUI will be displayed with a few options for the user to change such as the volume of the music/sound, timer on/off, light/dark mode

Quit Game - all processes will be halted, and the game will close.

Each GUI will have an options menu where the settings can be accessed and a "Quit" button that will take the user back to the main menu.



Functional Requirements

All project requirements will be written using the user story format which has the template:

As a *(stakeholder)*, I want to *(goal)* so that *(motivation)*.

Each story will focus on a specific testable requirement. Requirements will be represented in a logical order of importance. The user stories are listed here in a convenient table. Please read the table according to the template above.

Stakeholder	Goal	Motivation
user	solve inputted sudoku boards	I can solve boards I am stuck with
user	play sudoku	I can enjoy this classic game on my computer
user	access a large variety of random boards	I can continue to play new boards
user	see the rules of sudoku	can learn the rules of the game
user	be able to input guesses, into the corner of a square, and final answers, into the middle of the square, to sudoku squares	play the game as I normally would on paper with guesses and final answers formatted the same.
user	check my answers against the solution	I can see if I successfully solved the puzzle
user	save my board progress	I can continue that board when I return to the game
board maker	be able to input my homemade boards	I can see if they are legal and solvable
user	be able to save my progress in the game using a username and password	so that I can see boards I have already solved and continue

Non-Functional Requirements

The primary NFRs for this project are reliability, usability, availability. These three NFR are particularly important to the project to ensure we meet our goal of creating a streamlined, accessible, and easy-to-use application to solve sudoku.

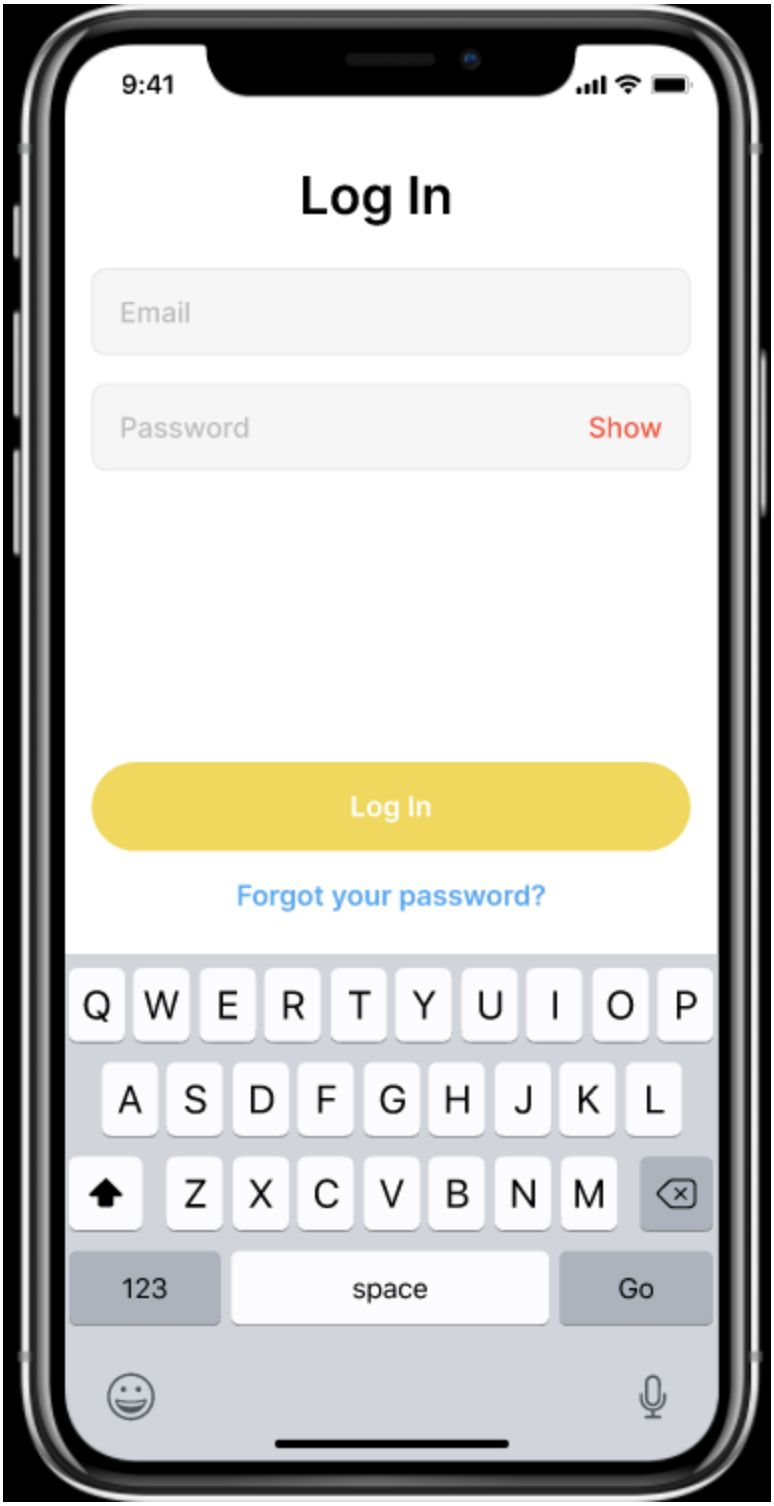
In order to meet these NFRs, extra care and attention must be paid when making decisions that potentially impact the layout, organization or style of the user interface. The layout of buttons and menus should be intuitive and organized, with appropriate color contrast and text size to ensure usability. The team will also ensure that the program is developed in such a way that is reliable and able to handle errors, whether caused by user or program, with ease. The program should be compatible with multiple platforms and operating systems to ensure the program is available for users should they need to use it. The program will also operate in a timely manner with short loading screens and little delay between menus and solution checking.

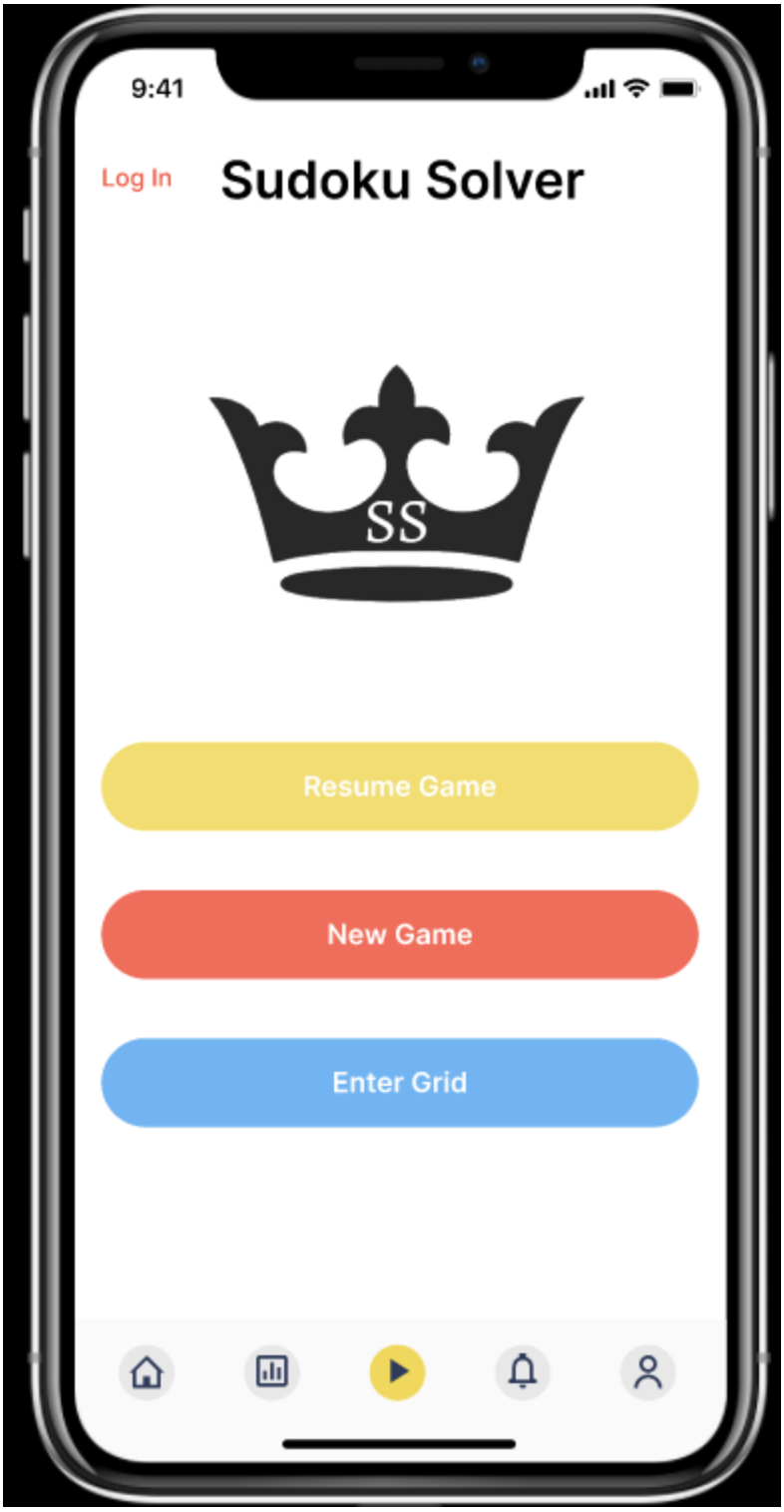
The secondary NFRs for this project are security, regulatory and data integrity. These requirements are essential to ensure that the users data is stored safely and securely, and that all pertaining data and privacy regulations are adhered to.

In order to meet these secondary requirements the data stored by the app will be stored in such a way that it complies with all necessary Canadian rules and regulations.

The layout of the GUI

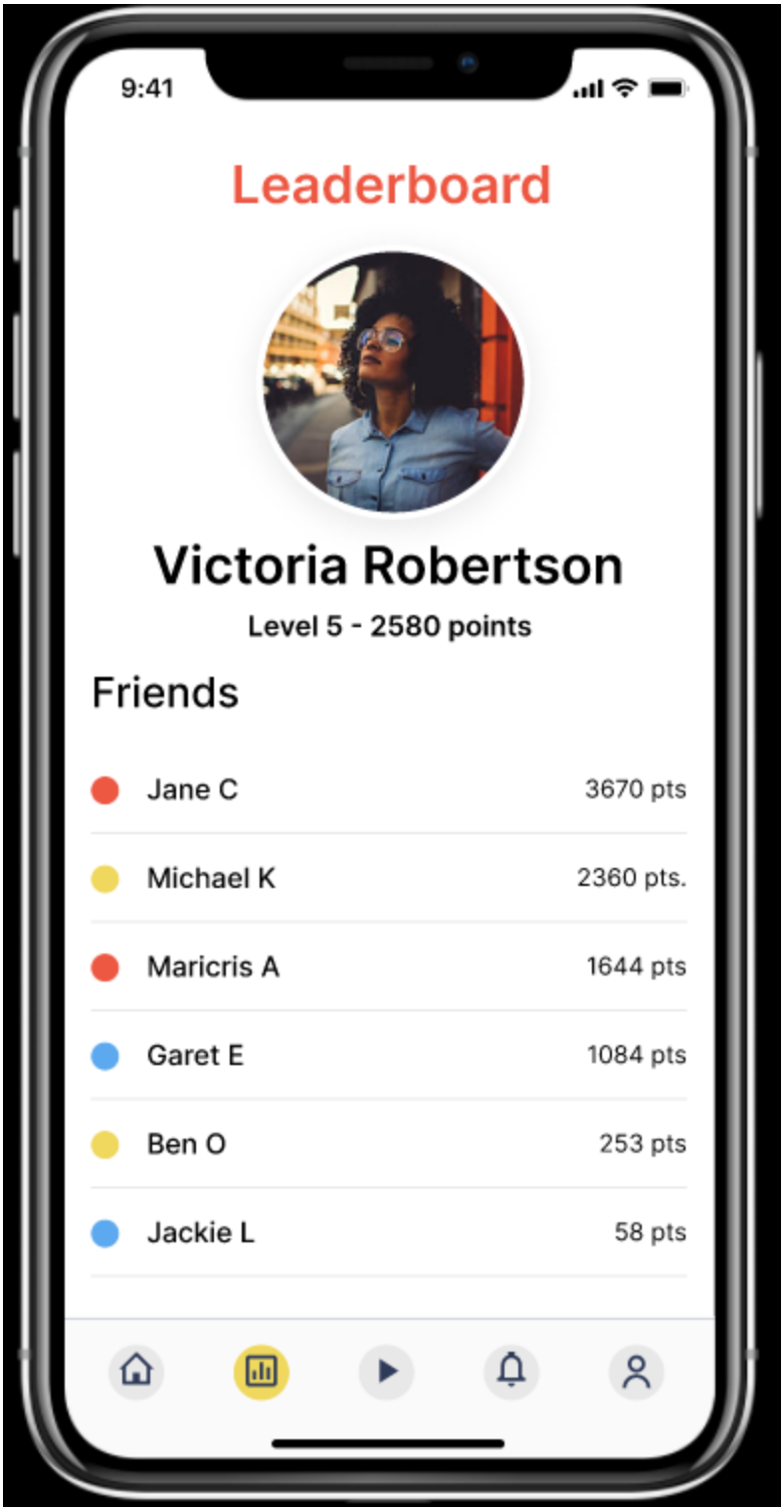
The prototype of the GUI contains 5 screens: the login screen, play screen, sudoku board screen, profile screen, and leaderboard screen. For the sake of this document the sudoku solver's GUI will be presented in application format but the final product will likely be a computer application. The GUI layout will be nearly identical but just formatted to fit on a large screen.


9:41
Log In
Email
Password
Show
Log In
Forgot your password?
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M
123
space
Go


9:41
Log In
Sudoku Solver
SS
Resume Game
New Game
Enter Grid

9:41
Back
1 8 7 4
5 9 8
6 9 2
5 2
6 3 4 9
6
6
2 7 5
9 8 4
1
2 ABC
3 DEF
4 GHI
5 JKL
6 MNO
7 PQRS
8 TUV
9 WXYZ
0

9:41
Settings
Profile
Logout
Victoria Robertson
Level 5 - 2580 points
Past Games
Scans
Easy
8m ago
Completed in 05:30
Hard
October 3
Completed in 23:07
Scanned Grid
September 21
Completed in 12:56



Screen #1 - Log In

The user is prompted to enter their email and password to gain access to the application. Upon pressing the yellow "Log In" button, the user will move to Screen #2. There is also the option to hide or show the user's password when entering, and an option to retrieve a user's password from the database if forgotten.

Screen #2 - Play screen

This screen displays the application logo, along with three action options for the user to choose from. If the first button, "Resume Game" is pressed, a previous but incomplete game will load and allow the user to continue play. If the second button, "New Game" is pressed, the user will be provided with an in-app game to play. Lastly if the third button, "Enter Grid" is pressed the user will be prompted to enter the grid manually. This functionality may also be extended to scan a grid through the smartphone camera as described in the project scope.

On the bottom of the screen, a tab menu is seen. There are five logos displayed, which have the functionalities listed below:

- House - home screen
- Bar Graph - leaderboard
- Play button - play
- Bell - notifications
- User - profile

Screen #3 - Game Screen

On this screen, a sudoku grid is shown. Below, there is a number pad to input numbers to play the game and above, there is a small tab menu. There are four buttons to choose from:

- Arrow - undo the last move
- Question mark - provide a hint
- Plus sign - start a new game
- Gear - game settings

There is also a "Back" button which would return the user back to Screen #2 if pressed, and a moon logo in the top right corner which enables the user to toggle dark mode.

Screen #4 - Profile Screen

This screen displays the current user's profile, including name, profile photo, skill level, and the number of points acquired through gameplay. Below, a summary of recent games is displayed with details such as difficulty level, time to complete, and if the game was a scanned grid. This bar also toggles to show past scanned grids.

In the upper part of the screen, there are two actions that can be taken, the user may log out of their current account, or they can access their profile settings.

Once again, the tab menu is shown at the bottom of the screen.

Screen #5 - Leaderboard

This last screen shows a leaderboard option to the game as a potential option for the team to pursue. The current user's data is shown (profile photo, skill level, number of points), along with the point data of their "friends" who also play through the application. These tabs are not used as buttons, rather they only show the data.

The bottom tab menu is once again shown.

Assumptions and Constraints

The Sudoku application has a few assumptions to the code. A sudoku board created by the computer will only have one right answer for each board. A board cannot have two different answers. With this being the case, as soon as an integer is placed in the wrong spot there will be a feature showing it is an incorrect placeholder. All the numbers (1-9) have to be used in each 3x3 section of the 9x9 board. All the numbers(1-9) have to be used for each vertical and horizontal line. This applies to both the solve mode and the game mode.

For an original 9x9 Sudoku board, it will be constrained to only integers (eg. 1-9), not letters. However, for a 16x16 Sudoku board, they can contain letters on the Sudoku board. The scan function that will be used in the solve mode will be constrained to only the sudoku board, therefore it will not be able to scan other objects.

Roles

The team roles were established through discussion of past experience and the way in which team members participated in meetings.

Tristan - Team Manager
Ian - Software Architect
Ben - Repository Design / Maintenance
Niko - Archivist
Camila - Artist
Nicholas - Public Relations
Yexin - Technical Writer
All Team Members - Testing

The project has been broken into four sections that can be worked on simultaneously. From there, each team member has been assigned their main coding role.

Graphical User Interface
- Yexin
- Camila

Login System
- Ian
- Niko

Sudoku Board Scanner
- Ben

Solver Algorithm
- Nicholas
- Tristan